

Dynamic Drawer System

Developer Reference Guide

Version	Audience	Last Updated
1.0.0	Frontend Developers	June 2026

This document explains how the centralised drawer system works in BusinessSAAS, how to add new feature drawers, and the conventions every developer must follow.

1. Overview

The Dynamic Drawer System provides a single, shared slide-in panel that any page or component can open with any content. Instead of each feature building its own drawer (with duplicated animation, overlay, and keyboard handling), every drawer in the application is the same shell — only the inner content changes.

Without this system	With this system
TaskDrawer.tsx — GSAP code + form LeadDrawer.tsx — GSAP code + form ContactDrawer.tsx — GSAP code + form 10 files, same animation bug in 10 places	Drawer.tsx — GSAP once, forever TaskForm.tsx — just the form LeadForm.tsx — just the form 1 bug fix fixes everything

2. Architecture

How the system is wired together:

```
OrgProvider.tsx
■■■ <DrawerProvider> ← mounts ONE <Drawer> shell for entire dashboard
■■■ <Sidebar>
■■■ <Topbar>
■■■ <main>
■■■ TasksPage
■■■ useDrawer() ← openDrawer({ title, content })
↓
DrawerContext ← stores config in state
↓
<Drawer> ← GSAP slides in with new content
■■■ <TaskForm /> ← just the form, no drawer code
```

File structure

```
src/
contexts/
DrawerContext.tsx ← Provider + useDrawer() hook
components/
ui/
Drawer.tsx ← Shell (animation, overlay, header, close)
tasks/
TaskForm.tsx ← Form only — no GSAP, no overlay
leads/
LeadForm.tsx ← Same pattern (future)
contacts/
ContactForm.tsx ← Same pattern (future)
```

3. API Reference

DrawerProvider

Wrap the dashboard shell once in `OrgProvider.tsx`. It mounts a single Drawer shell and provides context to all children.

```
import { DrawerProvider } from '@contexts/DrawerContext'

// In OrgProvider.tsx – wrap once
return (
  <DrawerProvider>
    <Sidebar />
    <main>{children}</main>
  </DrawerProvider>
)
```

useDrawer() hook

Call from any client component inside the dashboard.

```
import { useDrawer } from '@contexts/DrawerContext'

const { openDrawer, closeDrawer } = useDrawer()
```

Method	Parameters	Description
<code>openDrawer(config)</code>	DrawerConfig	Slides the drawer open with given content
<code>closeDrawer()</code>	—	Slides the drawer closed and clears content

DrawerConfig interface

```
interface DrawerConfig {
  title: string // Header text
  content: React.ReactNode // Your form component
  width?: 'sm' | 'md' | 'lg' // 360 | 440 | 560px (default: 'md')
}
```

Width value	Pixels	Use case
<code>'sm'</code>	360px	Simple forms (1-2 fields)
<code>'md'</code>	440px	Standard forms — default
<code>'lg'</code>	560px	Complex forms, rich text, file uploads

4. How to Add a New Feature Drawer

Follow these 3 steps every time you need a new drawer. Do not copy the Drawer shell. Do not add GSAP to your form.

01

Create the Form component

Create a new file in `src/components/{feature}/`. It contains ONLY the form. No overlay, no GSAP, no close button in the header. Use `useDrawer()` to call `closeDrawer()` when done.

```
// src/components/leads/LeadForm.tsx
'use client'

import { useDrawer } from '@contexts/DrawerContext'

interface LeadFormProps {
  lead?: Lead // undefined = create, Lead = edit
  onSave: (values: LeadFormValues) => Promise<void>
}

export default function LeadForm({ lead, onSave }: LeadFormProps) {
  const { closeDrawer } = useDrawer()

  const onSubmit = async (values: LeadFormValues) => {
    await onSave(values) // parent handles API call
    closeDrawer() // form closes itself on success
  }

  return (
    <div className="flex flex-col h-full">
      <form className="flex-1 overflow-y-auto px-6 py-5">
        { /* your fields here */ }
      </form>
      <div className="border-t px-6 py-4">
        <button onClick={closeDrawer}>Cancel</button>
        <button type="submit">Save</button>
      </div>
    </div>
  )
}
```

02

Open the drawer from your page

Import `useDrawer` in your page component. Call `openDrawer()` with a title, the form component as content, and an `onSave` callback that handles the API call and state update.

```
// src/app/(dashboard)/[orgId]/crm/leads/page.tsx

import { useDrawer } from '@contexts/DrawerContext'
import LeadForm from '@components/leads/LeadForm'
import { createLead } from '@lib/leads'
```

```

export default function LeadsPage() {
  const { openDrawer } = useDrawer()
  const [leads, setLeads] = useState<Lead[]>([])

  const handleCreate = () => {
    openDrawer({
      title: 'New lead',
      width: 'md',
      content: (
        <LeadForm
          onSave={async (values) => {
            const lead = await createLead(orgId, values)
            setLeads(prev => [lead, ...prev])
            // closeDrawer() is called inside LeadForm
          }}
        />
      ),
    })
  }

  return <button onClick={handleCreate}>New lead</button>
}

```

03

That's it

DrawerProvider is already mounted in OrgProvider. No configuration, no registration, no new providers. The drawer is ready.

5. Do's and Don'ts

DO

- ✓ Put only form logic in your Form component. No GSAP, no overlay, no escape key handler.
- ✓ Call `closeDrawer()` inside your Form's `onSubmit` — after the API call succeeds.
- ✓ Pass `onSave` as a prop. The page owns API calls and state updates, not the form.
- ✓ Use `width='lg'` for complex forms with many fields or rich text editors.
- ✓ Import `useDrawer()` anywhere inside the dashboard — it is always available.

DON'T

- ✗ Never import `Drawer.tsx` or `DrawerContext.tsx` directly in a Form component.
- ✗ Never add GSAP animations inside a Form component — the shell handles all motion.
- ✗ Never call `openDrawer()` outside the dashboard (it needs `DrawerProvider` as an ancestor).
- ✗ Never put API calls inside `Drawer.tsx` or `DrawerContext.tsx` — they are UI only.
- ✗ Never nest drawers. One drawer at a time. Close the current one before opening another.

6. Responsibilities at a Glance

File	Responsible for	NOT responsible for
<code>DrawerContext.tsx</code>	open/close state, config storage	animations, API calls, form logic
<code>Drawer.tsx</code>	GSAP slide, overlay, ESC key, header	content, form state, API calls
<code>XxxForm.tsx</code>	form fields, validation, <code>onSave</code> call	animations, overlay, close button header
<code>XxxPage.tsx</code>	API calls, state updates, <code>onSave</code> prop	form rendering, animation, close logic

7. Troubleshooting

Problem	Likely cause	Fix
"useDrawer must be used inside DrawerComponent"	Component is outside <code>OrgProvider</code> or in <code>auth/login</code> page	Only call <code>useDrawer()</code> in (dashboard) pages

Drawer opens but content is blank	content prop is undefined or null	Check openDrawer() call — ensure content: <YourForm>
Drawer does not animate on first open	GSAP init useEffect ran before refs were attached	Ensure Drawer.tsx is always in the DOM (DrawerProvider)
Form submits but drawer stays open	closeDrawer() not called after onSave	Call closeDrawer() inside Form onSubmit after await onSave